



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Final Report Document
Versione 1.0

Team

Choaib Goumri NF22500223
Mattia Gallucci NF22500127

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
1.1	Sintesi dei Traguardi Raggiunti	1
2	Contesto e Baseline del Sistema	2
2.1	Architettura CLI/GUI	2
2.2	Architettura WebApp (gateway-services)	2
2.3	Modalità di utilizzo	2
3	Change Requests e Analisi di Impatto	3
3.1	CR01 - Stabilità e Accessibilità dell'Ambiente (Docker)	3
3.2	CR02 - Progress Bar CLI	4
3.3	CR03 - Smell: Randomness Uncontrolled	4
3.4	CR04 - Smell: Hyperparameter not explicitly set	4
3.5	CR05 - Metrica: "Densità degli Smell"	5
3.6	Analisi di Impatto:	5
3.6.1	Sintesi dell'Analisi di Impatto	5
4	Strategia di Testing e Risultati	6
4.1	Master Test Plan	6
4.2	Pre-modification Testing	6
4.3	Post-modification Testing	7
4.4	Regression Testing	8
5	Conclusioni e Sviluppi Futuri	9
5.1	Sviluppi Futuri	9

1 Introduzione

Il presente elaborato rappresenta il rapporto conclusivo relativo al progetto CodeSmile, illustrando in dettaglio il percorso di sviluppo software intrapreso. Verranno esaminate le fasi salienti che hanno caratterizzato l'evoluzione dell'architettura, partendo dalla baseline iniziale fino all'integrazione delle modifiche richieste (Change Requests CR01-CR05). Scopo primario è offrire una panoramica esaustiva e documentata delle attività di analisi d'impatto e delle sessioni di verifica (testing), a testimonianza della qualità e della robustezza del prodotto finale.

1.1 Sintesi dei Traguardi Raggiunti

L'iter progettuale si è concluso con successo, permettendo il rilascio di una build di sistema stabile e ricca di funzionalità aggiuntive. Tra i risultati più significativi conseguiti dal team si annoverano:

- **Portabilità e Containerizzazione:** Risoluzione delle criticità legate alla configurazione ambientale, garantendo la piena compatibilità tra esecuzione locale e container Docker senza necessità di interventi manuali sui file sorgente (CR01).
- **Interfaccia Utente e Feedback:** Implementazione di una barra di avanzamento dinamica per la CLI, incrementando la reattività percepita e l'usabilità durante l'analisi di repository di grandi dimensioni (CR02).
- **Potenziamento Analisi ML:** Estensione del motore di detection con nuovi controlli mirati alla riproducibilità degli esperimenti (Randomness - CR03) e alla validazione esplicita degli iperparametri nei modelli (CR04), aumentando l'affidabilità del codice scientifico.
- **Metriche di Qualità Avanzate:** Integrazione dell'indice "Densità degli Smell" nei report generati, fornendo un parametro statistico normalizzato per valutare lo stato di salute del codice in rapporto alla sua dimensione (CR05).
- **Affidabilità e Validazione:** Esecuzione con esito positivo di una suite completa di test (pass rate 100%), comprendente verifiche di regressione sull'intero sistema e test End-to-End automatizzati per la WebApp.

2 Contesto e Baseline del Sistema

CodeSmile è uno strumento di static code analysis orientato all'individuazione di ML-specific code smells in progetti Python. Il sistema combina regole basate su AST con componenti di orchestrazione (CLI/GUI) e una WebApp a microservizi (gateway, AI Analysis, Static Analysis, Report). La baseline del progetto è composta da due macro-ambienti: (i) l'esecuzione locale via CLI/GUI, e (ii) la fruizione via WebApp. L'architettura a package autonomi riduce l'accoppiamento e abilita interventi evolutivi mirati, mantenendo tracciabilità tra requisiti, componenti e test.

2.1 Architettura CLI/GUI

Il core locale comprende la pipeline di analisi orchestrata dal runner, i componenti di analisi (Inspector e RuleChecker), gli extractor semantici e il modulo di reportistica. I package principali includono: `cli`, `components`, `detection_rules`, `code_extractor` e `report`.

2.2 Architettura WebApp (gateway-services)

La WebApp segue un'architettura gateway-services basata su FastAPI. Il gateway espone gli endpoint e inoltra le richieste ai servizi: AI Analysis Service, Static Analysis Service e Report Service. Ogni servizio mantiene una struttura a livelli (main, routers, utils) per separare avvio, API e logica applicativa.

2.3 Modalità di utilizzo

- **CLI:** analisi di singoli file o progetti, con opzioni di parallelismo (walkers), gestione input/output e ripresa (resume).
- **GUI:** configurazione guidata e visualizzazione log in tempo reale.
- **WebApp:** upload di file o progetti e consultazione dei risultati (lista smell e report).

3 Change Requests e Analisi di Impatto

Questa sezione descrive le modifiche apportate al sistema attraverso le Change Requests (CR) pianificate. Per ogni CR è stata condotta un'analisi di impatto basata su approccio white-box (Call Graph e Matrici di Raggiungibilità) per definire gli insiemi di impatto SIS (Start Impact Set), CIS (Candidate Impact Set) e confrontarli con l'AIS (Actual Impact Set) post-implementazione.

Di seguito una sintesi delle Change Requests implementate:

ID	Titolo	Categoria	Esito
CR01	Stabilità e Accessibilità (Docker)	Preventive	Rilasciata
CR02	Progress Bar CLI	Perfective	Rilasciata
CR03	Randomness Uncontrolled	Perfective	Rilasciata
CR04	Hyperparameter not explicitly set	Perfective	Rilasciata
CR05	Metrica: "Densità degli Smell"	Perfective	Rilasciata

3.1 CR01 - Stabilità e Accessibilità dell'Ambiente (Docker)

Obiettivo: Rendere il codice agnostico rispetto all'ambiente di esecuzione (Locale vs Container), eliminando la gestione manuale degli import e degli endpoint (hardcoded) attraverso l'adozione del pattern Externalized Configuration.

Analisi di Impatto: L'analisi statica ha identificato il Gateway come nodo centrale (Orchestrator) e i servizi backend come componenti passivi.

- **SIS (Start Impact Set):** 9 file identificati (main entry points e router dei servizi).
- **CIS (Candidate Impact Set):** Coincidente con il SIS, data l'assenza di ripple effects previsti.
- **AIS (Actual Impact Set):** 9 file effettivamente modificati.

Metriche:

- Precision: 1.00
- Recall: 1.00

3.2 CR02 - Progress Bar CLI

Obiettivo: Introdurre una barra di avanzamento visiva nella CLI per fornire feedback in tempo reale durante l'analisi di progetti di grandi dimensioni, migliorando la User Experience (UX).

Analisi di Impatto: È stata analizzata l'interazione tra `cli_runner` e `project_analyzer`. È emersa la necessità di un meccanismo di callback per notificare il progresso.

- **SIS:** `project_analyzer.py` e `requirements.txt`.
- **AIS:** Coincidente con il SIS.

Metriche:

- Precision: 1.00
- Recall: 1.00

3.3 CR03 - Smell: Randomness Uncontrolled

Obiettivo: Implementare una regola di analisi statica per identificare la mancata impostazione del seed (random state) negli algoritmi di Machine Learning, garantendo la riproducibilità degli esperimenti.

Analisi di Impatto: Trattandosi di una modifica evolutiva su architettura a plugin, l'impatto è stato confinato alla creazione della nuova regola.

- **SIS:** `rule_checker.py` e il nuovo file `randomness_uncontrolled.py`.
- **AIS:** Coincidente con il SIS.

Metriche:

- Precision: 1.00
- Recall: 1.00

3.4 CR04 - Smell: Hyperparameter not explicitly set

Obiettivo: Rilevare l'utilizzo implicito dei valori di default per iperparametri critici dei modelli ML, per prevenire rischi di "Silent Failure" dovuti all'aggiornamento delle librerie.

Analisi di Impatto: L'analisi prevedeva modifiche alla configurazione dei modelli e alla logica di controllo.

- **SIS:** `models.csv`, nuova regola `hyperparameters...py`, `rule_checker.py`.
- **AIS:** Durante l'implementazione è emerso che `rule_checker.py` non necessitava modifiche, mentre è stato necessario intervenire su `model_extractor.py` (DIS - Discovered Impact Set).

Metriche:

- Precision: 0.67 (Sovrastima del coinvolgimento di `rule_checker`)
- Recall: 0.67 (Mancata previsione di `model_extractor`)

3.5 CR05 - Metrica: "Densità degli Smell"

Obiettivo: Introdurre un indicatore statistico (Smell Density) che normalizza il numero di difetti sulle righe di codice (LOC) per migliorare l'interpretabilità dei report.

Analisi di Impatto: La modifica è trasversale (Cross-Cutting), coinvolgendo l'estrazione (Inspector), l'aggregazione e il reporting.

- **SIS:** 8 file, inclusi frontend (`page.tsx`), tipi, API, inspector e report generator.
- **AIS:** L'implementazione ha rivelato la necessità di modificare anche i contratti API e i router del servizio di analisi statica (`detect_smell.py`, `responses.py`, `static_analysis.py`), inizialmente non previsti. Alcuni moduli di estrazione previsti nel SIS non hanno invece richiesto modifiche.

Metriche:

- Precision: 0.75
- Recall: 0.67

3.6 Analisi di Impatto:

La modifica è trasversale (Cross-Cutting), coinvolgendo l'estrazione (Inspector), l'aggregazione e il reporting.

- **SIS:** 8 file, inclusi frontend (`page.tsx`), tipi, API, inspector e report generator.
- **AIS:** L'implementazione ha rivelato la necessità di modificare anche i contratti API e i router del servizio di analisi statica (`detect_smell.py`, `responses.py`, `static_analysis.py`), inizialmente non previsti.

3.6.1 Sintesi dell'Analisi di Impatto

Di seguito si riporta la tabella di sintesi che confronta gli insiemi di impatto stimati ed effettivi, evidenziando le metriche di accuratezza (Precision e Recall) per ciascuna Change Request.

CR	SIS	CIS	AIS	Precision	Recall
CR01 (Docker)	9 file	9 file	9 file	1.00	1.00
CR02 (Progress Bar)	2 file	2 file	2 file	1.00	1.00
CR03 (Randomness)	2 file	2 file	2 file	1.00	1.00
CR04 (Hyperparameter)	3 file	3 file	3 file	0.67	0.67
CR05 (Smell Density)	8 file	8 file	9 file	0.75	0.67

4 Strategia di Testing e Risultati

Il processo di verifica e validazione è stato strutturato in conformità con il Master Test Plan, coprendo quattro livelli fondamentali: test di unità, integrazione, sistema e testing End-to-End (E2E). La baseline per la regressione è stata stabilita attraverso test di sistema pre-modifica basati sul metodo Category Partition.

4.1 Master Test Plan

La strategia di testing adottata ha previsto:

- **Unit Testing:** Approccio white-box focalizzato sulla struttura interna del codice, con un obiettivo di branch coverage superiore all'80% per i moduli critici.
- **Integration Testing:** Approccio incrementale per verificare le interazioni tra UI, Inspector, RuleChecker e Report Generator.
- **System Testing:** Approccio black-box basato sul comportamento osservabile dall'utente, utilizzando il metodo Category Partition per derivare i casi di test.
- **E2E Testing:** Automatizzazione dei flussi utente sulla WebApp tramite Cypress.

4.2 Pre-modification Testing

Prima dell'implementazione delle CR, è stata eseguita una suite di System Test (TC_01 - TC_20) derivata tramite Category Partition. I parametri considerati per la generazione dei test frame includono:

- **Input e Struttura:** Numero di file, Estensione (.py), Struttura directory (lineare/annidata).
- **Smell:** Numero (0, 1, >1) e Tipologia (Generico, API-Specific).
- **Configurazione Tool:** Modalità (CLI/GUI), Parallelismo, Walkers, Resume, Output path.
- **Infrastruttura Web:** Disponibilità backend ed esito Gateway.

4.3 Post-modification Testing

Le attività di testing post-modifica hanno verificato specificamente le componenti introdotte dalle cinque Change Requests.

- **System Test Estes:** È stata introdotta una nuova categoria "Livello di Densità / Qualità" nel Category Partition, generando nuovi casi di test (TC_21 - TC_24) per validare la CR05 (Smell Density).
- **End-to-End Testing (Cypress):** È stata implementata una suite di 27 test E2E distribuiti su 7 file di specifica, coprendo:
 - Navigazione e Routing (Homepage).
 - Upload e Analisi (File singolo e Progetti multi-file).
 - Reportistica e visualizzazione delle metriche di qualità.
- **Verifiche CR specifiche:**
 - **CR01:** Verifica avvio ibrido Locale/Container e iniezione variabili d'ambiente.
 - **CR02:** Verifica sincronizzazione progress bar CLI e overhead nullo.
 - **CR03/CR04:** Verifica positive/negative detection per le nuove regole (Randomness e Hyperparameters).
 - **CR05:** Validazione matematica del calcolo densità e propagazione nel report JSON/Excel.

4.4 Regression Testing

La regressione ha previsto la riesecuzione completa di tutte le suite di test (Backend, Frontend, System) per garantire l'assenza di effetti collaterali indesiderati.

- **Copertura del Codice:** 90% complessiva, 100% sui file modificati/nuovi.
- **Esito Complessivo:** Tutti i test eseguiti hanno dato esito positivo (Pass Rate 100%).

Di seguito il riepilogo dell'esecuzione della Test Suite:

Categoria di test	Tipo	Totale	Superati	Falliti	Tasso di successo
Unit Tests	CLI Components	16	16	0	100%
Unit Tests	Code Extractors	43	43	0	100%
Unit Tests	Core Components	57	57	0	100%
Unit Tests	Detection Rules (API)	27	27	0	100%
Unit Tests	Detection Rules (Generic)	78	78	0	100%
Unit Tests	GUI Components	55	55	0	100%
Unit Tests	Report Generation	44	44	0	100%
Integration Tests	Component Integration	7	7	0	100%
Integration Tests	Workflow Integration	1	1	0	100%
System Tests	Full System Test	24	24	0	100%
Web App	Jest	27	27	0	100%
Web App	Cypress	27	27	0	100%
Totale		312	312	0	100%

Tabella 4.1: Riepilogo esecuzione Test Suite

5 Conclusioni e Sviluppi Futuri

Il progetto CodeSmile ha completato con successo il ciclo di evoluzione pianificato, consolidando un processo strutturato basato su change management, impact analysis rigorosa e quality assurance a più livelli. Le cinque Change Requests implementate (CR01-CR05) hanno aumentato significativamente la portabilità del sistema (Docker support), l'usabilità per l'utente finale (Progress Bar e Metriche di Densità) e la capacità di analisi specifica per il dominio Machine Learning (regole per riproducibilità e iperparametri).

La fase di testing, culminata con l'esecuzione di 406 test totali, ha confermato la stabilità della baseline (100% Pass Rate) e l'efficacia delle nuove funzionalità.

5.1 Sviluppi Futuri

Per le prossime iterazioni del progetto, sono state identificate tre aree strategiche di miglioramento, volte a trasformare CodeSmile da semplice strumento di rilevamento a piattaforma di supporto attivo allo sviluppo:

1. Analisi Storica e Attribuzione ("Git Blame Integration")

- **Obiettivo:** Introdurre un modulo che, interfacciandosi con la history della repository (es. Git), identifichi non solo *dove* si trova uno smell, ma *chi* e *quando* lo ha introdotto.
- **Valore:** Questo permetterà di contestualizzare il debito tecnico e fornire feedback mirati agli sviluppatori, facilitando percorsi di formazione specifici.

2. Proposta di Soluzione Assistita da AI

- **Obiettivo:** Integrare un motore (eventualmente basato su LLM leggeri o pattern matching avanzato) che non si limiti a segnalare l'errore, ma proponga attivamente uno snippet di codice correttivo.

3. CodeSmile Assistant

- **Obiettivo:** Sviluppare un tool interattivo (chatbot o wizard CLI) a supporto dell'utente stesso di CodeSmile.
- **Funzionalità:** L'assistente guiderà l'utente nella configurazione dell'analisi (scelta dei walker, selezione regole) e nell'interpretazione dei report, democratizzando l'uso dello strumento anche per team meno esperti nell'ambito della static analysis.